

Containerizing and Monitoring the ShopSphere Microservices Platform

Business Scenario

The ShopSphere platform has successfully evolved into a cloud-native microservices architecture consisting of independently deployable Spring Boot services. The platform now includes a centralized **Spring Cloud Config Server**, **Eureka Service Discovery**, an **API Gateway**, and a reusable **Common Security Module** that provides consistent JWT-based authentication and authorization across all business services.

As the platform moves closer to production deployment, the DevOps and Operations teams have identified several operational challenges that must be addressed before the application can be deployed consistently across multiple environments.

Currently, each microservice is executed directly from the development environment using an IDE or command-line interface. Starting the complete platform requires developers to manually launch every service in the correct order, configure environment-specific properties, and ensure supporting services such as the database and RabbitMQ are available. This manual deployment process is difficult to reproduce across development, testing, and production environments.

To simplify deployment and ensure consistency, every ShopSphere microservice should be packaged as an independent Docker container. Each service must be capable of running independently using its own Docker image, allowing individual deployment, testing, and version management.

Once individual containerization has been completed, the architecture team wants to automate the deployment of the complete ShopSphere platform using **Docker Compose**. Rather than manually starting each container, Docker Compose should orchestrate the startup of all business services and supporting infrastructure as a single application stack, ensuring the correct startup sequence and service dependencies.

In addition to deployment improvements, the Operations team requires better visibility into the running applications. Every microservice should generate meaningful application logs to assist in troubleshooting, auditing, and operational analysis. Logging should capture important business operations, security events, application lifecycle events, and unexpected failures while avoiding the exposure of sensitive information.

The platform should also expose runtime health and monitoring information using **Spring Boot Actuator**. Operations teams should be able to verify service availability, application health, runtime information, and basic application metrics without accessing the application internals.

The enhanced ShopSphere platform should demonstrate how enterprise Spring Boot microservices are packaged, deployed, monitored, and maintained using containerization

and built-in monitoring capabilities while preserving the distributed architecture implemented in previous scenarios.

TODOs

Docker Overview

Containerization Fundamentals

- Understand the purpose of containerization.
 - Compare virtual machines and Docker containers.
 - Understand Docker Images and Docker Containers.
 - Review the benefits of Docker for microservices.
-

Dockerizing Individual Microservices

Creating Dockerfiles

Create an individual Dockerfile for each Spring Boot application.

Applications include:

- Spring Cloud Config Server
- Eureka Discovery Server
- API Gateway
- Order Service
- Inventory Service
- Notification Service

Each Dockerfile should:

- Package the Spring Boot executable JAR.
 - Configure the runtime environment.
 - Expose the required application port.
 - Start the application automatically.
-

Building Docker Images

Build Docker images independently for each service.

Verify:

- Image creation
 - Image versioning
 - Image naming conventions
-

Running Individual Containers

Run each Docker image independently.

Validate:

- Application startup
- Port mapping
- Environment configuration
- Container logs

Verify that each service can execute independently inside its own container.

Docker Compose

Multi-Container Deployment

Create a Docker Compose configuration for the complete ShopSphere platform.

The deployment should include:

- Config Server
 - Eureka Discovery Server
 - API Gateway
 - Order Service
 - Inventory Service
 - Notification Service
 - Database
 - RabbitMQ
-

Docker Compose Configuration

Configure:

- Service definitions
- Port mappings
- Container names
- Environment variables
- Service dependencies
- Shared Docker network
- Persistent storage where required

Ensure that all services can communicate through the Docker network.

Platform Deployment

Deploy the complete application stack using Docker Compose.

Verify:

- Startup order
 - Successful service registration
 - Successful configuration retrieval
 - Gateway routing
 - Application availability
-

Application Logging

Logging Fundamentals

- Understand the importance of logging in distributed applications.
- Configure logging using **SLF4J** with Spring Boot's default logging framework.
- Review different logging levels.

Use appropriate log levels:

- TRACE
- DEBUG

- INFO
- WARN
- ERROR

Business Logging

Log important application events including:

- Application startup
- User authentication
- Order creation
- Order updates
- Inventory validation
- Notification processing
- Service startup and shutdown
- Exception handling

Avoid logging:

- Passwords
- JWT secrets
- Sensitive customer information

Exception Logging

Generate meaningful log messages for:

- Validation failures
- Authentication failures
- Authorization failures
- Service exceptions
- Unexpected runtime errors

Ensure exceptions are logged using the Global Exception Handler implemented in previous scenarios.

Application Monitoring

Spring Boot Actuator

Enable Spring Boot Actuator for every microservice.

Configure Actuator endpoints to expose:

- Application Health
- Application Information
- Metrics
- Environment Details
- Bean Information
- Request Mappings

Health Monitoring

Verify application health using Actuator.

Monitor:

- Service availability
- Database connectivity
- RabbitMQ connectivity
- Application readiness
- Application liveness

Runtime Monitoring

Use Actuator to monitor:

- JVM memory usage
- Thread information
- HTTP request metrics
- Application uptime
- System information

Verify that monitoring information is available for every microservice.

Deployment Validation

Validate the complete platform by verifying:

Individual Docker Deployment

- Each service builds successfully as a Docker image.
 - Each service runs independently inside its own container.
 - Container logs are available.
 - Application starts successfully.
-

Docker Compose Deployment

- Entire platform starts successfully using Docker Compose.
 - Config Server starts before client services.
 - Eureka registers all services.
 - API Gateway routes requests successfully.
 - Database connectivity is established.
 - RabbitMQ connectivity is established.
-

Logging Validation

Verify:

- Application startup logs.
 - Business operation logs.
 - Security logs.
 - Exception logs.
 - Container logs.
-

Monitoring Validation

Verify:

- Health endpoint.
- Information endpoint.
- Metrics endpoint.
- Runtime statistics.
- Service Availability

Use the previous project which is created in lab day 7 ,do not create the project from scratch